SUNWAY UNIVERSITY

CSC3209 - Software Architecture and Design Pattern – Practical 8_Sample Solution

Consider the following software system:

A bank wants to build a software system for ATM machine to allow users to do the following:

- Insert an ATM card.
- Eject an ATM card
- Insert Pin Number
- Withdraw money

The scenarios for this system are as follows:

1- When the ATM machine has no inserted card, the machine should act as follows:
   a. When the user inserts a card, the ATM machine should request a pin number from the user.
   b. When the user requests to eject a card, the machine should inform the user that there is no inserted card.
   c. When the user tries to enter a pin number, the ATM machine should inform user that there is no inserted card.
   d. When the user requests to withdraw money, the ATM machine should inform user that there is no inserted card.

2- When the ATM machine has inserted card, the machine should act as follows:
   a. When the user tries to insert a card, the ATM machine should inform the user that there is already an inserted card in the machine.
   b. When the user requests to eject a card, the machine should eject the card and inform the user that the card has been ejected.
   c. When the user tries to enter a pin number, the ATM machine should inform user that the entered number is correct when it is correct, otherwise it should inform the user that the entered number is wrong and eject the card.
   d. When the user requests to withdraw money, the ATM machine should inform user that there is no entered pin number.

3- When the ATM machine has inserted card and the pin number has been entered correctly, the machine should act as follows:
   a. When the user tries to insert a card, the ATM machine should inform the user that there is already an inserted card in the machine.
   b. When the user requests to eject a card, the machine should eject the card and inform the user that the card has been ejected.
   c. When the user tries to enter a pin number, the ATM machine should inform user that the pin number has been entered correctly already.

d. When the user requests to withdraw money, the ATM machine should provide money to user and eject the card if the requested amount is less than the available money in the machine. Otherwise, the ATM machine should inform the user that there is no enough money and eject the card if the requested amount is more than the available money in the machine.

4- When the ATM machine does not have cash money, the machine should act as follows:
   a. When the user inserts a card, the ATM machine should inform the user that there is no money in the ATM machine and eject the card.
   b. When the user requests to eject a card, the machine should inform the user that there is no money in the ATM machine and there is no inserted card to eject.
   c. When the user tries to enter a pin number, the ATM machine should inform user that there is no money in the ATM machine.
   d. When the user requests to withdraw money, the ATM machine should inform user that there is no money in the ATM machine.

Propose a suitable design patten to address the system scenarios and write its corresponding code.

Sample Answer:

```
package ATMMachineStateExample;
public class ATMMachine {
	ATMState hasCard;
	ATMState noCard;
	ATMState hasCorrectPin;
	ATMState atmOutOfMoney;
	ATMState atmState;

	int cashInMachine = 3000;
	boolean correctPinEntered = false;
	public ATMMachine(){
		hasCard = new HasCard(this);
		noCard = new NoCard(this);
		hasCorrectPin = new HasPin(this);
		atmOutOfMoney = new NoCash(this);
		atmState = noCard;
		if(cashInMachine < 0){
			atmState = atmOutOfMoney;
		}
	}

	void setATMState(ATMState newATMState){
		atmState = newATMState;
	}

	public void setCashInMachine(int newCashInMachine){
		cashInMachine = newCashInMachine;
	}

	public void insertATMCard() {
		atmState.insertATMCard();
	}

	public void ejectATMCard() {
		atmState.ejectATMCard();
```

```
	}

	public void withdrawCash(int cashToWithdraw) {
		atmState.withdrawCash(cashToWithdraw);
	}

	public void insertPin(int pinEntered){
		atmState.insertPin(pinEntered);
	}

	public ATMState getYesCardState() { return hasCard; }
	public ATMState getNoCardState() { return noCard; }
	public ATMState getHasPin() { return hasCorrectPin; }
	public ATMState getNoCashState() { return atmOutOfMoney; }
}
```

```
package ATMMachineStateExample;
public interface ATMState {
	// Different states of ATM machine
	// HasCard, NoCard, HasPin, NoCash
	void insertATMCard();
	void ejectATMCard();
	void insertPin(int pinEntered);
	void withdrawCash(int cashToWithdraw);
}
```

```
package ATMMachineStateExample;
public class NoCard implements ATMState {

	ATMMachine atmMachine;
	public NoCard(ATMMachine newATMMachine){
		atmMachine = newATMMachine;
	}

	@Override
	public void insertATMCard() {
		System.out.println("Please enter your pin");
		atmMachine.setATMState(atmMachine.getYesCardState());
	}

	@Override
	public void ejectATMCard() {
		System.out.println("You didn't enter a card");
	}

	@Override
	public void withdrawCash(int cashToWithdraw) {
		System.out.println("You have not entered your card");
	}

	@Override
	public void insertPin(int pinEntered) {
		System.out.println("You have not entered your card");
	}
```

```
}
```

```
package ATMMachineStateExample;
public class HasCard implements ATMState {

	ATMMachine atmMachine;
	public HasCard(ATMMachine newATMMachine){
		atmMachine = newATMMachine;
	}

	@Override
	public void insertATMCard() {
		System.out.println("You can only insert one card at a time");
	}

	public void ejectATMCard() {
		System.out.println("Your card is ejected");
		atmMachine.setATMState(atmMachine.getNoCardState());
	}

	@Override
	public void withdrawCash(int cashToWithdraw) {
		System.out.println("You have not entered your PIN");
	}

	@Override
	public void insertPin(int pinEntered) {
		if(pinEntered == 1234){
			System.out.println("You entered the correct PIN");
			atmMachine.correctPinEntered = true;
			atmMachine.setATMState(atmMachine.getHasPin());

		} else {
			System.out.println("You entered the wrong PIN");
			atmMachine.correctPinEntered = false;
			System.out.println("Your card is ejected");
			atmMachine.setATMState(atmMachine.getNoCardState());
		}
	}
}
```

```
package ATMMachineStateExample;
public class HasPin implements ATMState {
	ATMMachine atmMachine;
	public HasPin(ATMMachine newATMMachine){
		atmMachine = newATMMachine;
	}
	@Override
	public void insertATMCard() {
		System.out.println("You already inserted a card");
	}

	@Override
	public void ejectATMCard() {
```

```
            System.out.println("Your card is ejected");
            atmMachine.setATMState(atmMachine.getNoCardState());
      }

      @Override
      public void withdrawCash(int cashToWithdraw) {
            if(cashToWithdraw > atmMachine.cashInMachine){
                  System.out.println("There isn't enough available cash");
                  System.out.println("Your card is ejected");
                  atmMachine.setATMState(atmMachine.getNoCardState());

            } else {
                  System.out.println(cashToWithdraw+" is provided by the
                  machine");
                  atmMachine.setCashInMachine(atmMachine.cashInMachine -
                                              cashToWithdraw);

                  System.out.println("Your card is ejected");
                  atmMachine.setATMState(atmMachine.getNoCardState());

                  if(atmMachine.cashInMachine <= 0){
                     atmMachine.setATMState(atmMachine.getNoCashState());

                  }
            }
      }

      @Override
      public void insertPin(int pinEntered) {
            System.out.println("You already entered a PIN");
      }
}
```

```
package ATMMachineStateExample;
public class NoCash implements ATMState {

      ATMMachine atmMachine;
      public NoCash(ATMMachine newATMMachine){
            atmMachine = newATMMachine;
      }

      @Override
      public void insertATMCard() {
            System.out.println("There is no money in this ATM");
            System.out.println("Your card is ejected");
      }

      @Override
      public void ejectATMCard() {
            System.out.println("There is no money in this ATM");
            System.out.println("There is no card to eject");
      }

      @Override
      public void withdrawCash(int cashToWithdraw) {
            System.out.println("There is no money in this ATM");
```

```
	}

	@Override
	public void insertPin(int pinEntered) {
		System.out.println("There is no money in this ATM");
	}
}
```

```
package ATMMachineStateExample;
public class TestATMMachine {

	public static void main(String[] args){
		ATMMachine atmMachine = new ATMMachine();
		atmMachine.insertATMCard();
		atmMachine.ejectATMCard();
		atmMachine.insertATMCard();
		atmMachine.insertPin(1234);
		atmMachine.withdrawCash(3000);
		atmMachine.insertATMCard();
		atmMachine.insertPin(1234);
	}
}
```

Output:

```
Please enter your pin
Your card is ejected
Please enter your pin
You entered the correct PIN
3000 is provided by the machine
Your card is ejected
There is no money in this ATM
Your card is ejected
There is no money in this ATM
```